Implementation of Battery Monitoring System via Modbus Protocol

Prepared by: Sara Bavi
Student ID: 427144

Rheinland-Pfälzische Technische Universität Kaiserslautern-Landau

December 19, 2025

Contents

1 Project Overview **3**

2 Battery Unit Description **4**

3 Communication Architecture and Network Discovery **4**

4 Data Verification (Modbus Poll) **6**

5 Data Processing and Visualization (HMI) **6**

6 Battery Configuration and CAN Communication **9**

7 Conclusion **11**

A Node-RED JSON Configuration **13**

B Modbus Register Map **15**

1 Project Overview

This report outlines the components, communication architecture, and data visualization platform used to monitor and configure the operational status of a battery energy storage unit. The objective of the system is to establish a link between the physical battery hardware and a digital monitoring interface, ensuring safe operation data analysis.

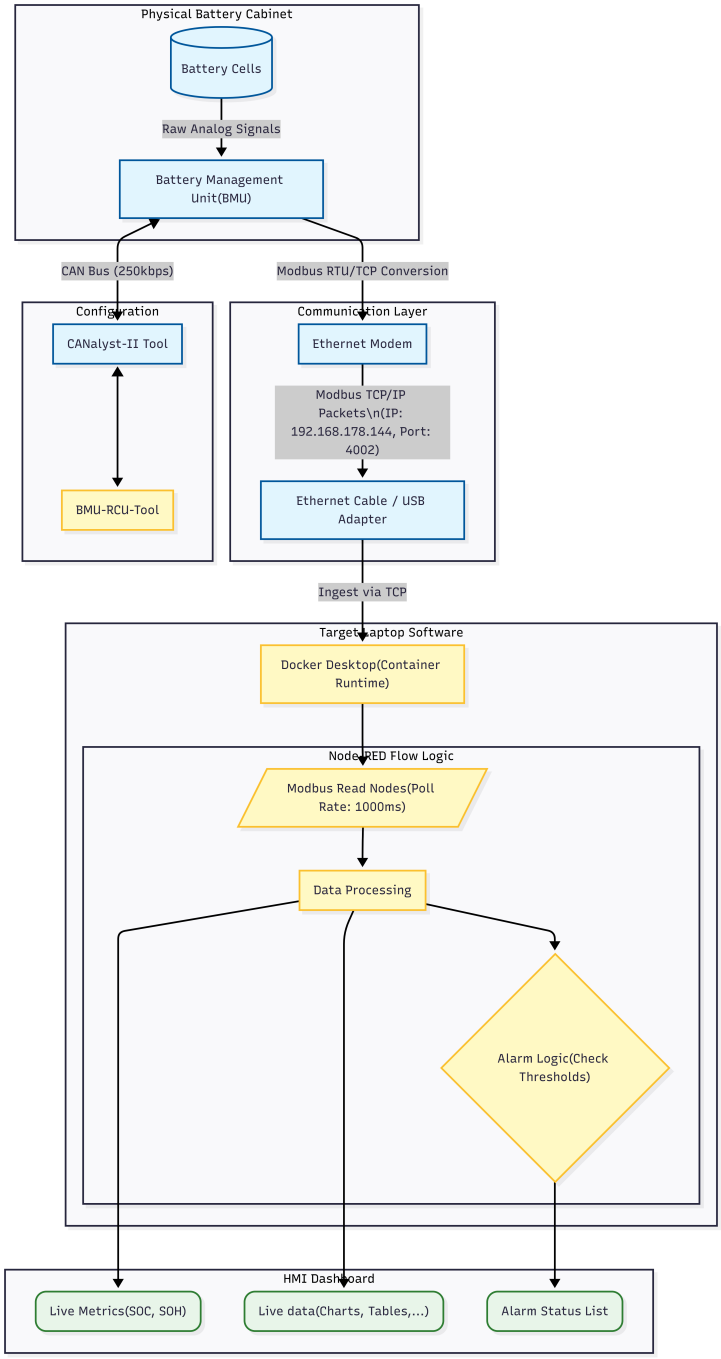

Figure 1: Complete System Architecture and Data Flow

2 Battery Unit Description

The physical architecture of the system is centered on a Battery Cabinet that houses the individual Battery Cells. To ensure safety and efficiency, the system utilizes two key control components:

- **Battery Controller Unit (BMU):** This unit handles the operational control and management of the system, acting as the primary Battery Management System (BMS) to monitor cell health.

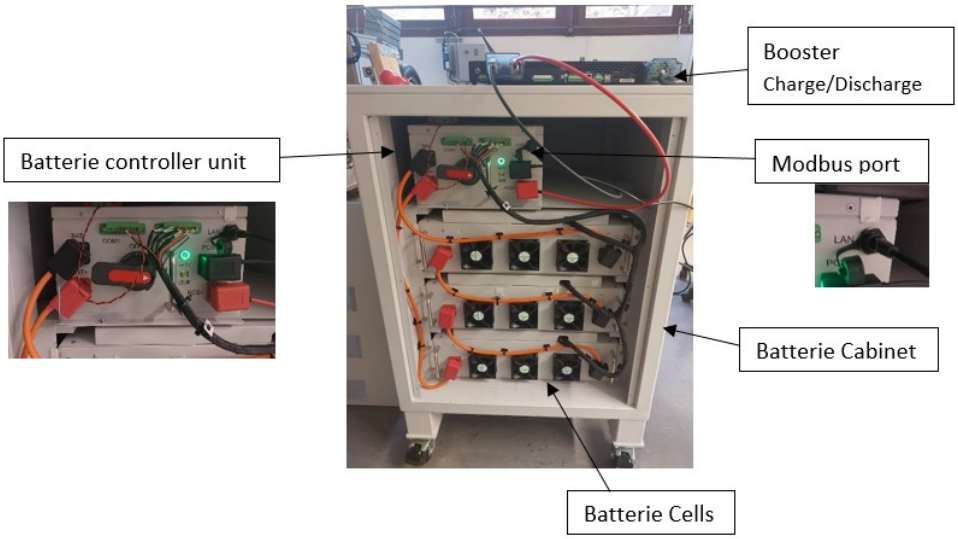

Figure 2: Battery Unit and Controller Components

- **Booster:** This component functions as a DC-DC converter, managing the critical Charge/Discharge functions to regulate power flow.

3 Communication Architecture and Network Discovery

The system employs a communication setup to link the battery controller to a host computer for monitoring. The data transmission path flows from the battery to an Ethernet Modem, connects via cable to a USB Adaptor, and finally terminates at the host computer.

The Role of Ethernet

In this project, Ethernet serves as the critical physical layer for the Modbus TCP/IP communication protocol. By utilizing an Ethernet Modem connected to the battery, the system leverages standard network infrastructure to transmit data reliably at high speeds. This setup allows the host computer to interface with the battery as a network node (IP: 192.168.178.144) rather than relying on short-range, point-to-point serial connections.

This architecture enhances scalability and simplifies the physical cabling required for monitoring.

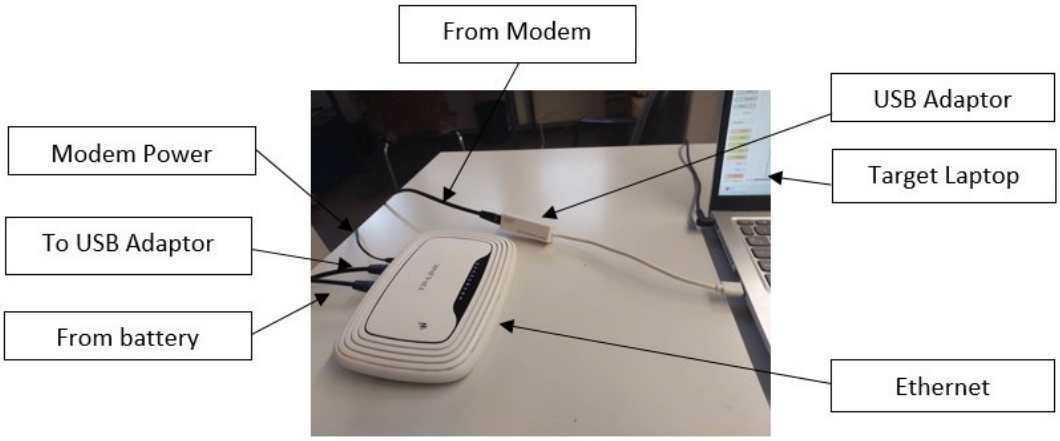

Figure 3: Communication Architecture Diagram

Network Analysis with Wireshark

To establish the initial connection parameters, the Wireshark network protocol analyzer was utilized. By placing the network interface in promiscuous mode, Wireshark allowed for the inspection of network packets to extract the destination IP address and listening port number. This analysis confirmed the target IP Address as **192.168.178.144** and the Port Number as **4002**. Once these parameters were verified via packet inspection, they were hardcoded into the application, eliminating the need for further analysis.

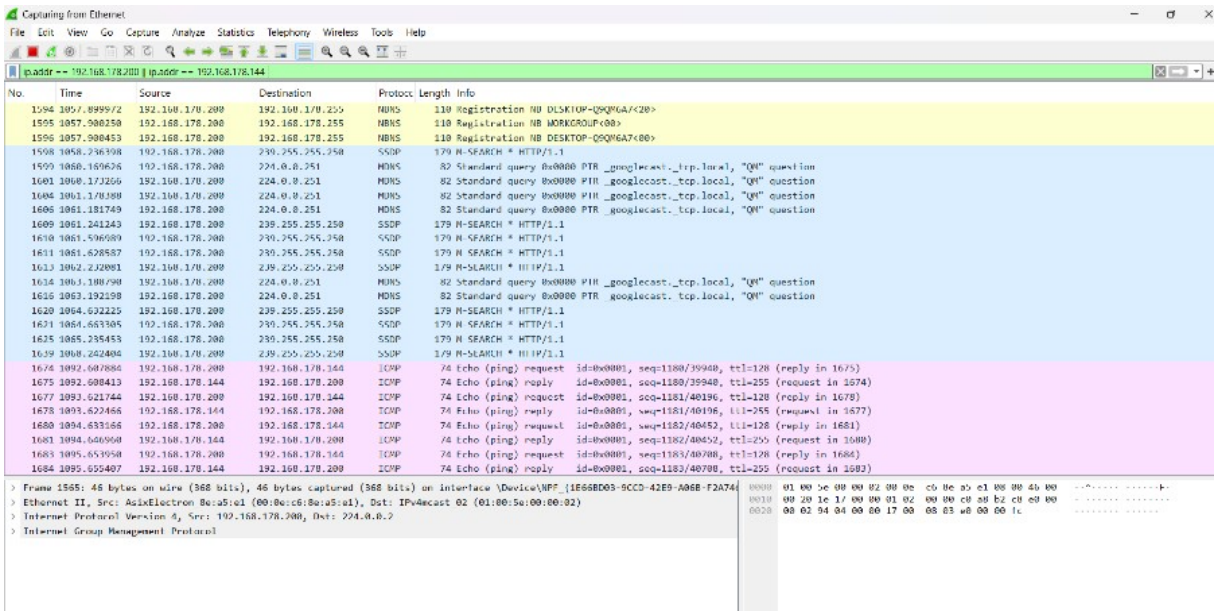

Figure 4: Network Analysis with Wireshark

4 Data Verification (Modbus Poll)

Before implementing the high-level dashboard, data integrity was confirmed using Modbus Poll, a Modbus Master simulator. This step was crucial for debugging communication errors and validating protocol compliance.

- **Connection:** The software connected directly to the battery's specific network address: (192.168.178.144): 4002.
- **Polling Strategy:** It employed standard Modbus functions, specifically Read Holding Registers (4X), to periodically retrieve data with a scan rate of 1000ms.

This verified that the slave device responded correctly to read/write commands before integration into the larger system.

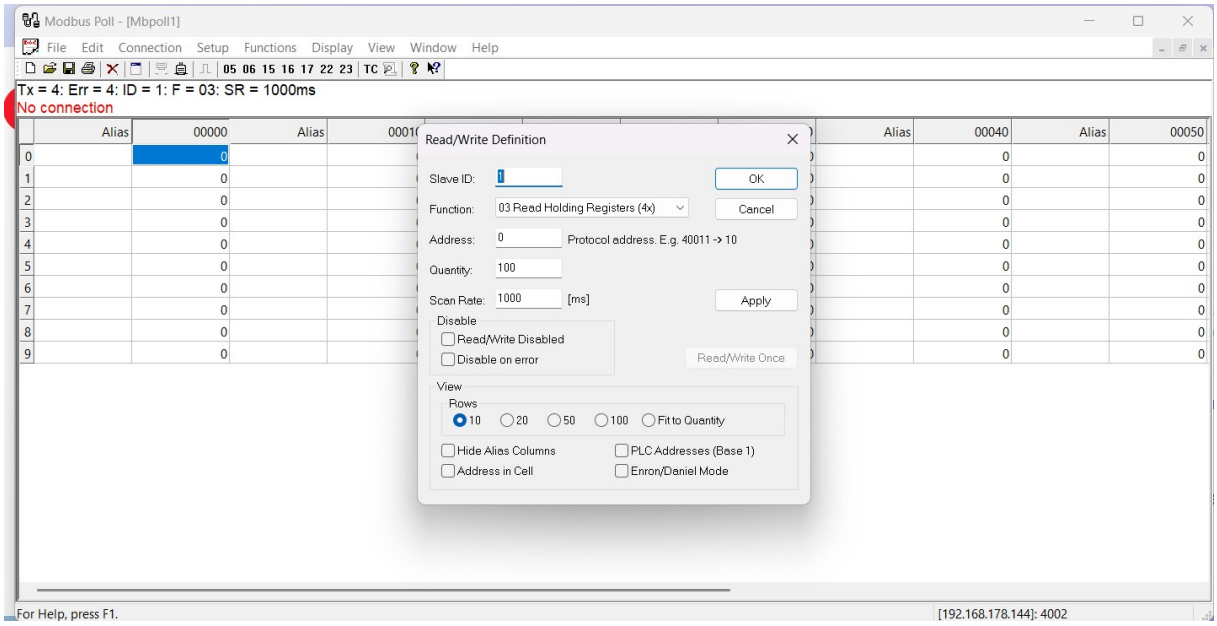

Figure 5: Data Verification using Modbus Poll

5 Data Processing and Visualization (HMI)

The monitoring data is processed and presented via a flow-based architecture deployed using containerization.

A. Data Processing (Node-RED)

The logic for handling incoming battery data is implemented using Node-RED. The Node-RED Flow manages the stages of data ingestion, processing, and routing. This flow is responsible for converting the raw Modbus data into actionable metrics for storage and visualization.

Flow Configuration Logic

The system logic is defined in a JSON structure. Below is a snippet of the code responsible for reading alarm data from the Modbus TCP server. The full configuration code is available in **Appendix A**.

Listing 1: Node-RED Modbus Read Configuration Snippet

```
{
    "id": "d3e76f5b2cd06f8e",
    "type": "modbus-read",
    "name": "Modbus Read - Alarms",
    "unitid": "1",
    "dataType": "Input",
    "adr": "200",
    "quantity": "152",
    "rate": "10",
    "rateUnit": "s",
    "server": "225e4eefb76f0e78"
}
```

B. Deployment Environment (Docker Desktop)

To ensure a consistent and portable software environment, Docker Desktop is used to manage the application. By running the software components in isolated Containers, the system ensures that the Node-RED runtime and its dependencies remain stable regardless of the host operating system configuration.

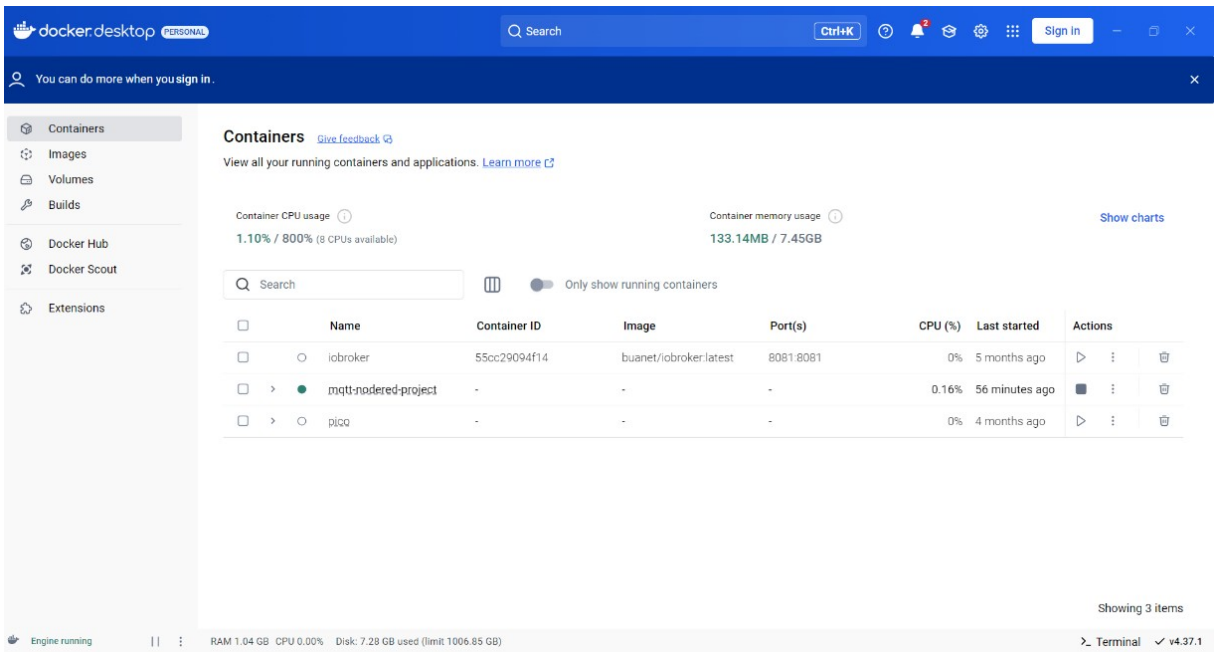

Figure 6: Deployment Environment (Docker Desktop)

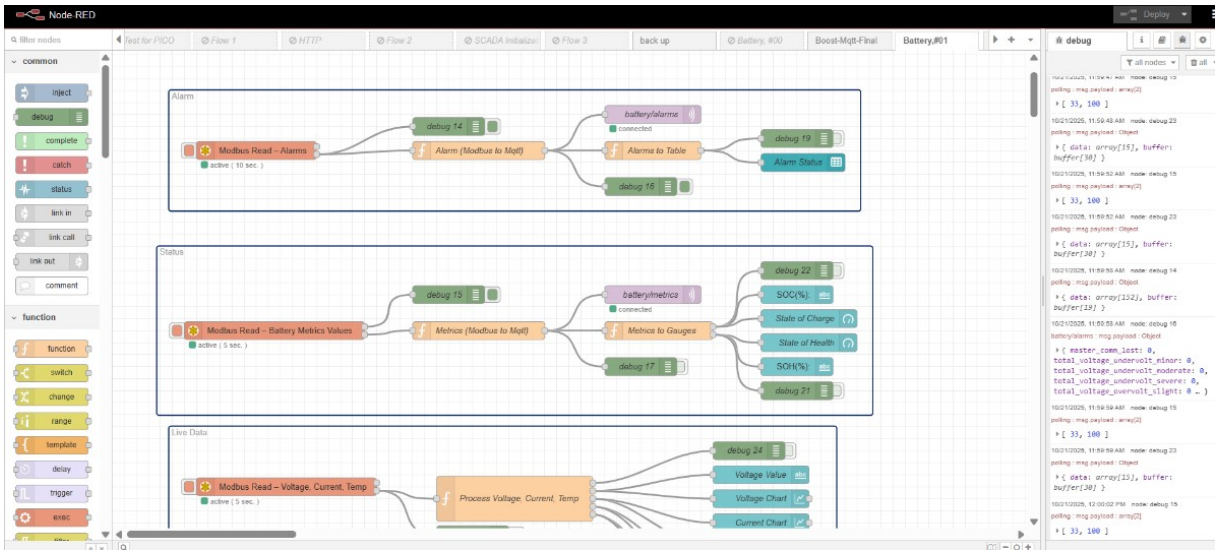

Figure 7: Node-RED Flow Logic

C. Human-Machine Interface (HMI)

The final user interface is the Battery Dashboard, which organizes processed data into readable panels. It provides operators with:

- **Live Data Values:** Readings of voltage and current.
- **Graphical Indicators:** Gauges for State of Charge (SOC) and State of Health (SOH).
- **Time-Series Charts:** Visual tracking of parameters over time.

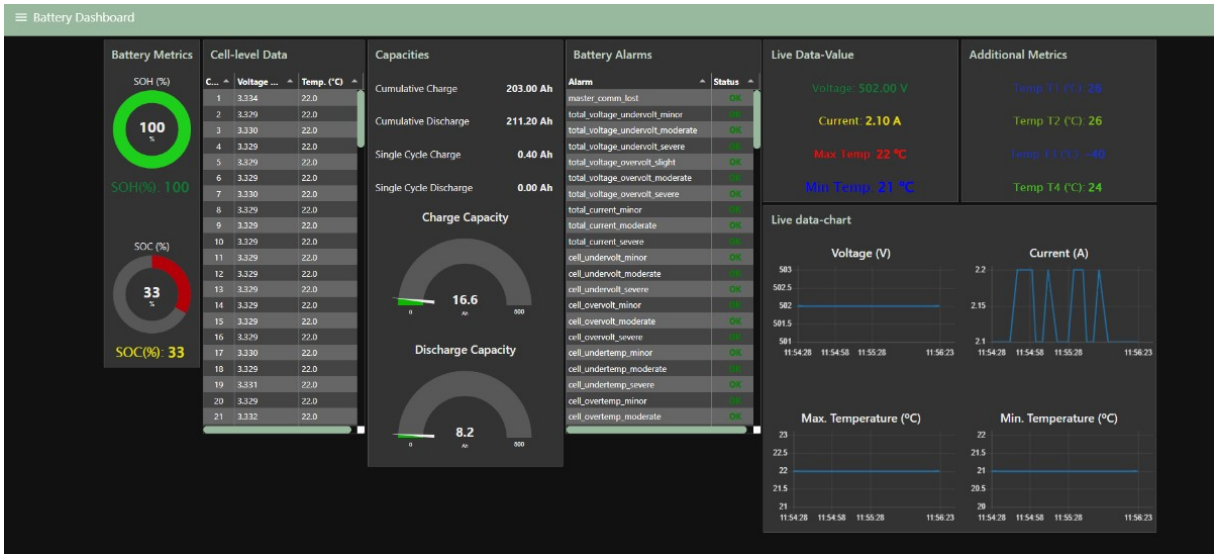

Figure 8: Battery Dashboard HMI

6 Battery Configuration and CAN Communication

While Modbus is used for monitoring, the system utilizes the Controller Area Network (CAN) protocol for low-level configuration.

CAN Protocol Specification

The Battery Management Unit (BMU) uses a CAN bus interface configured for a speed of 250 kbps (indicated as COM250K in the application setup). CAN is preferred in battery systems for its speed, reliability, and robust error detection, making it ideal for safety-critical component-level data exchange. The application uses a status of "Success Connection RCU", confirming a successful connection between the application and the Rack Control Unit (RCU).

Figure 9: CAN Bus Interface Hardware

Figure 10: CAN Communication Setup

BMS Configuration (BMU-RCU-Tool)

The dedicated application for managing and setting up the battery's operational limits is the BMU-RCU-Tool. This tool is essential for configuring the BMS (Battery Management System) functionality. Key functions of the BMU-RCU-Tool interface include:

- **Parameter Setting:** The Parameter tab contains critical settings for system safety and performance.
- **Threshold Configuration:** The tool allows operators to define and adjust thresholds for the BMU and RCU. This includes setting both Alarm values and Restore values for various operating conditions.
- **Voltage Limits:** Thresholds are set for cell and total battery over- and under-voltage conditions (e.g., Cell Overvol Alarm at 3.550V and Restore at 3.500V).
- **Temperature Limits:** Critical temperature thresholds are configured, such as Cell Over Temp (Alarm at 45.0°C, Restore at 40.0°C).
- **Alarm Monitoring:** The application categorizes and displays alarms into three levels (Level 1, Level 2, and Level 3).

The configuration interface ensures that the battery operates within safe parameters defined by the system designer, which is a core function of the BMS.

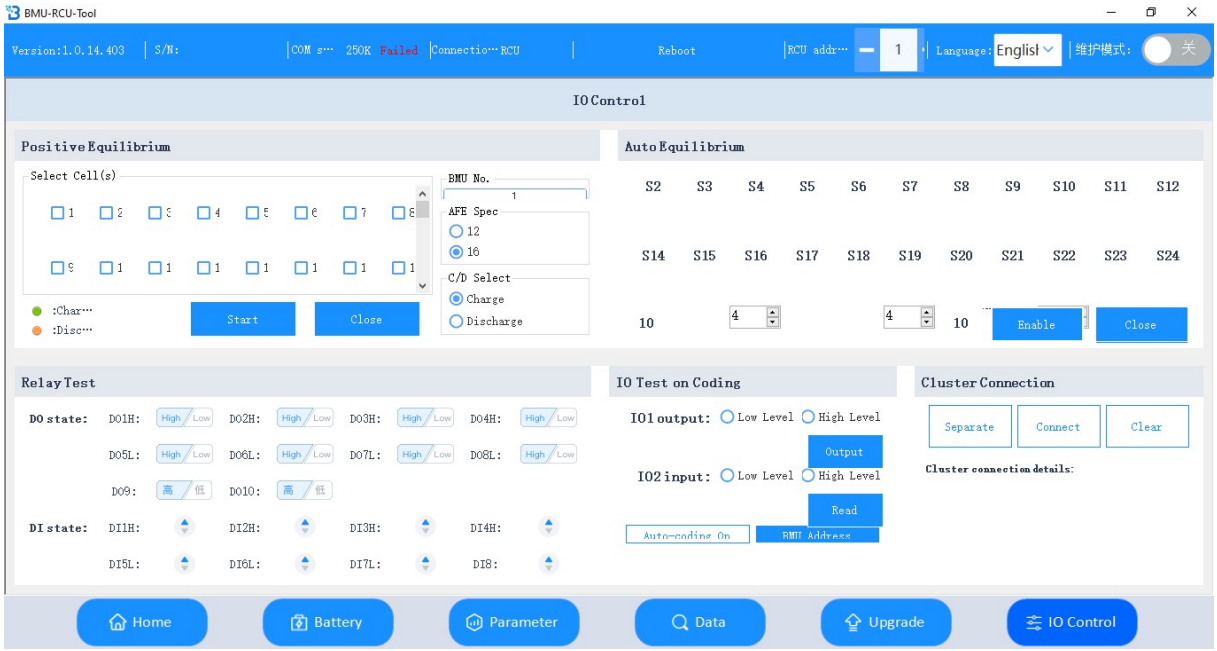

Figure 11: BMU-RCU-Tool Interface Overview

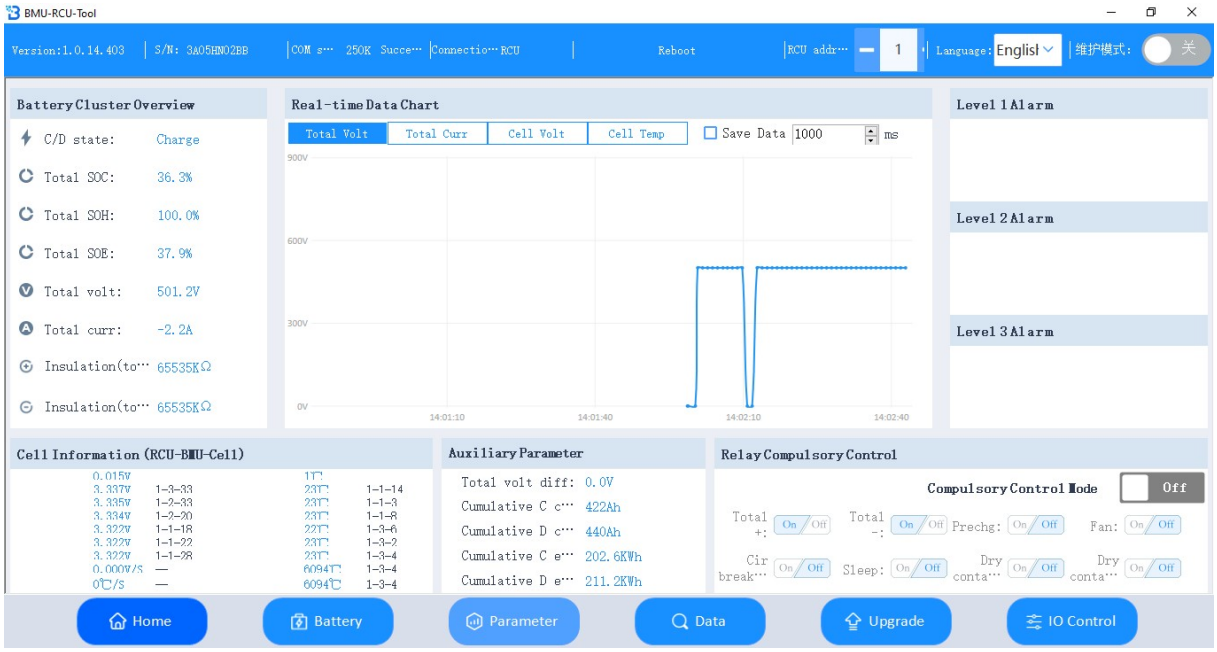

Figure 12: BMU-RCU-Tool Parameter Configuration

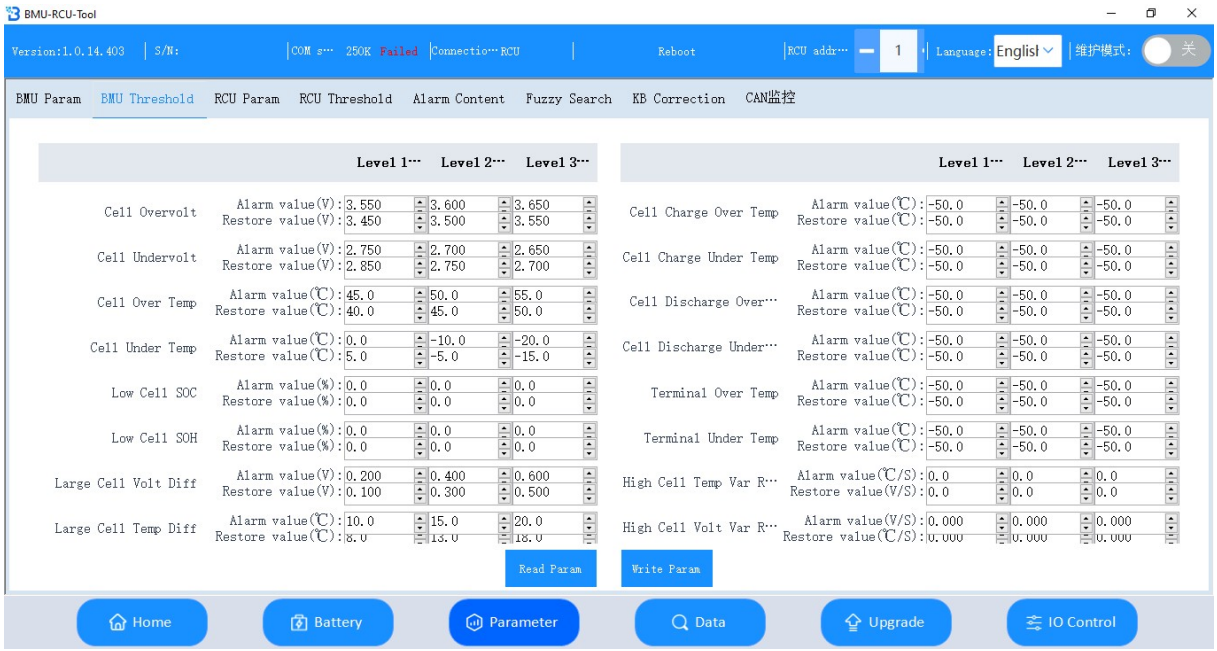

Figure 13: BMU-RCU-Tool Alarm and Threshold Settings

7 Conclusion

This project successfully demonstrated the implementation of a soft battery monitoring system. By integrating the physical Battery Management Unit with a Modbus TCP/IP communication layer, the system achieved reliable data acquisition and visualization. The use of Ethernet provided a robust network backbone, while the Node-RED and Docker environment ensured a scalable and flexible software architecture. Furthermore, the inclusion of the CAN protocol and the BMU-RCU-Tool enabled precise low-level

configuration and safety management. The resulting dashboard provides operators with critical insights into battery health (SOH) and charge status (SOC), ensuring the efficient and safe operation of the energy storage unit.

A Node-RED JSON Configuration

The following code represents the complete Node-RED flow configuration used to acquire, process, and visualize the battery data.


```
[
    {
        "id": "5d04b28e8fe4f3cf",
        "type": "tab",
        "label": "Battery,#01",
        "disabled": false,
        "info": "",
        "env": []
    },
    {
        "id": "cdc18140ff4d8090",
        "type": "group",
        "z": "5d04b28e8fe4f3cf",
        "name": "Alarm",
        "style": {
            "stroke": "#001f60",
            "label": true
        },
        "nodes": [
            "d3e76f5b2cd06f8e",
            "af9f8c61654d5499",
            "06f29b28005fd20a",
            "d7e30bd8123893d4",
            "d7453d27a094ff25",
            "849452d66f58832c",
            "cf53c02628291647",
            "b6b4e8ec678ee405"
        ],
        "x": 94,
        "y": 199,
        "w": 1152,
        "h": 202
    },
    {
        "id": "d3e76f5b2cd06f8e",
        "type": "modbus-read",
        "z": "5d04b28e8fe4f3cf",
        "g": "cdc18140ff4d8090",
        "name": "Modbus Read - Alarms",
        "unitid": "1",
        "dataType": "Input",
        "adr": "200",
        "quantity": "152",
        "rate": "10",
        "rateUnit": "s",
        "server": "225e4eefb76f0e78",
        "wires": [
            [],
            [
                "d7e30bd8123893d4",
                "af9f8c61654d5499"
            ]
        ]
    },
    {
        "id": "81140ea88ec49e41",
        "type": "modbus-read",
        "z": "5d04b28e8fe4f3cf",
        "g": "eea87e14efc9cc26",
        "name": "Modbus Read - Battery Metrics Values",
        "dataType": "InputRegister",
        "adr": "118",
        "quantity": "2",
        "rate": "5",
        "rateUnit": "s",
        "server": "225e4eefb76f0e78",
        "wires": [
            [
                "c5982b6251a95ef7"
            ],
            [
                "fd2130ed898d83e9"
            ]
        ]
    },
    {
        "id": "af9f8c61654d5499",
        "type": "function",
        "z": "5d04b28e8fe4f3cf",
        "g": "cdc18140ff4d8090",
        "name": "Alarm (Modbus to Mqtt)",
        "func": "let alarmNames = [\"master_comm_lost\", \"total_voltage_undervolt_minor\", \"
            total_voltage_undervolt_moderate\", \"total_voltage_undervolt_severe\", \"total_voltage_overvolt_slight\"
            , \"total_voltage_overvolt_moderate\", \"total_voltage_overvolt_severe\", \"total_current_minor\", \"
            total_current_moderate\", \"total_current_severe\", \"cell_undervolt_minor\", \"cell_undervolt_moderate\"
            , \"cell_undervolt_severe\", \"cell_overvolt_minor\", \"cell_overvolt_moderate\", \"cell_overvolt_severe
            \", \"cell_undertemp_minor\", \"cell_undertemp_moderate\", \"cell_undertemp_severe\", \"
            cell_overtemp_minor\", \"cell_overtemp_moderate\", \"cell_overtemp_severe\"];
```

```
// Simplified list for display",
        "outputs": 1,
        "wires": [
            [
                "06f29b28005fd20a",
                "d7453d27a094ff25",
                "cf53c02628291647"
            ]
        ]
    },
    {
        "id": "225e4eefb76f0e78",
        "type": "modbus-client",
        "name": "",
        "clienttype": "tcp",
        "tcpHost": "192.168.178.144",
        "tcpPort": "4002",
        "unit_id": 1,
        "clientTimeout": 1000,
        "reconnectOnTimeout": true
    }
]
```

Note: The full JSON is extensive. Key nodes for Modbus reading and configuration are included above.

B Modbus Register Map

The following table details the Modbus register addresses, their descriptions, sizes, and operational notes used for the battery system.

Address	Description	Size	Notes
100	Battery pack operating status	16-bit	0: Initial, 1: Charging, 2: Discharging, 3: Ready, 4: Maintenance, 5-7: Disabled, 8: Fault
101	Max allowed charging power	16-bit	Factor 0.1, Unit: kW
102	Max allowed discharging power	16-bit	Factor 0.1, Unit: kW
103	Max allowed charging voltage	16-bit	Factor 0.1, Unit: V
104	Max allowed discharging voltage	16-bit	Factor 0.1, Unit: V
105	Max allowed charging current	16-bit	Factor 0.1, Unit: A
106	Max allowed discharging current	16-bit	Factor 0.1, Unit: A
115	Pack voltage	16-bit	Factor 0.1, Unit: V
116	Pack current	16-bit	Factor 0.1, Unit: A, Offset -1600 A
117	Master module temperature	16-bit	Factor 1, Unit: °C, Offset -40°C
118	State of Charge (SOC)	16-bit	Unit: %, Factor 1
119	State of Health (SOH)	16-bit	Unit: %, Factor 1
120	Insulation resistance	16-bit	Factor 1000, Unit: Ohm
121	Average cell voltage	16-bit	Factor 0.001, Unit: V
122	Average cell temperature	16-bit	Factor 1, Unit: °C, Offset -40°C
123	Maximum cell voltage	16-bit	Factor 0.001, Unit: V
125	Minimum cell voltage	16-bit	Factor 0.001, Unit: V
127	Highest cell temperature	16-bit	Factor 1, Unit: °C, Offset -40°C
129	Lowest cell temperature	16-bit	Factor 1, Unit: °C, Offset -40°C
139	Total accumulated charge energy	32-bit	Factor 0.1, Unit: kWh
141	Total accumulated discharge energy	32-bit	Factor 0.1, Unit: kWh
191-690	Individual cell voltages (1-500)	16-bit	Factor 0.001, Unit: V
200	Master communication lost	1-bit	Alarm Status
201-203	Pack total undervoltage	1-bit	Minor/Moderate/Severe Alarm
204-206	Pack total overvoltage	1-bit	Slight/Moderate/Severe Alarm
207-209	Pack total current out of range	1-bit	Minor/Moderate/Severe Alarm
210-212	Cell undervoltage	1-bit	Minor/Moderate/Severe Alarm

Address	Description	Size	Notes
213-215	Cell overvoltage	1-bit	Minor/Moderate/Severe Alarm
216-218	Cell temperature too low	1-bit	Minor/Moderate/Severe Alarm
219-221	Cell temperature too high	1-bit	Minor/Moderate/Severe Alarm
891-1390	Individual cell temperatures (1-500)	16-bit	Factor 1, Unit: °C, Offset -40°C

Table 1: Selected Modbus Register Map